

Getting Started Guide

Edit New page

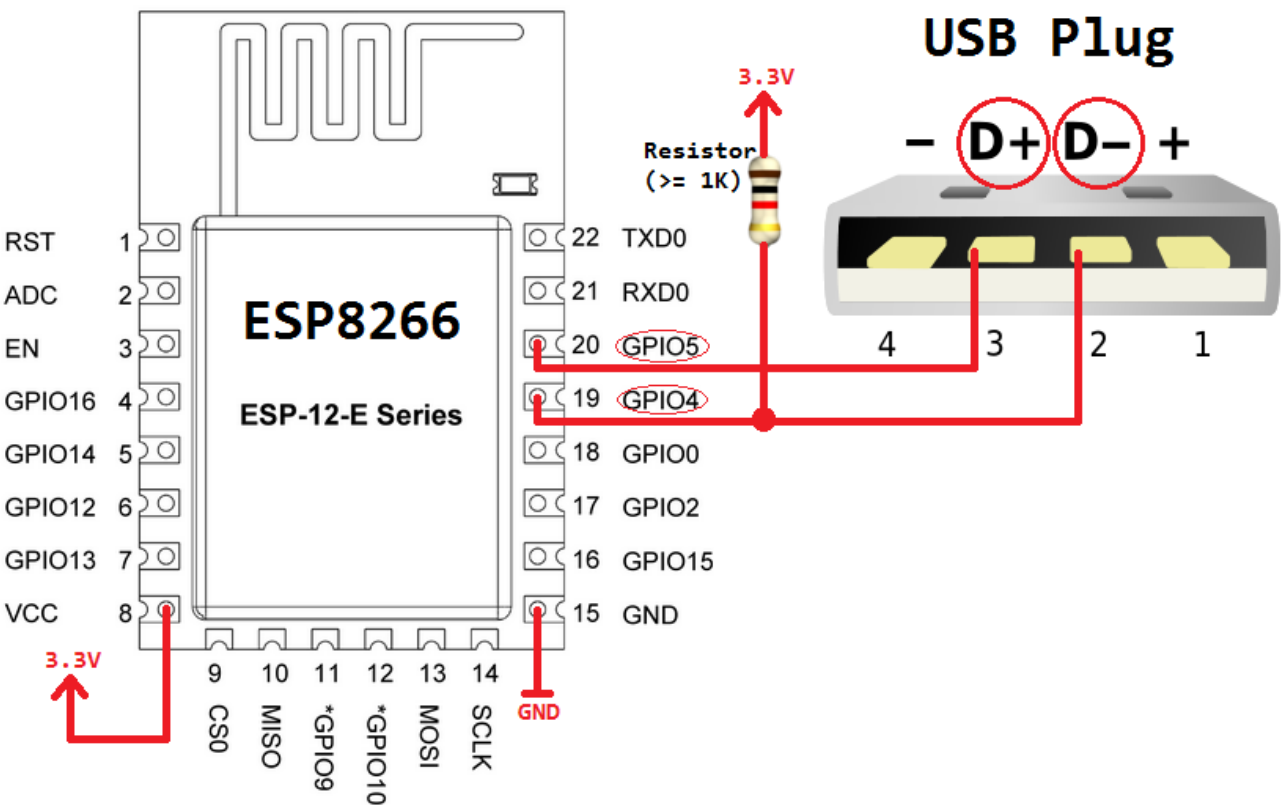
axell edited this page on Jan 2, 2024 · [38 revisions](#)

Table of Content

- [Hardware schematic diagram](#)
- [Building/Flashing using Espressif SDK](#)
 - [Building the toolchain](#)
 - [Building & flashing espusb](#)
 - [Editing user.cfg](#)
 - [Building website](#)
 - [Flash website over to ESP](#)
- [Building/Flashing using Docker container](#)
 - Install Docker and git
 - Flash ESP8266
- [Run HID example](#)

Hardware Schematics diagram

Below is the simplified version of the schematic diagram showing only the USB connection to the ESP8266.



Building/Flashing using Espressif SDK

Building the toolchain

This tutorial assumes that you haven't compiled your toolchains yet and it is a good start for new beginners. Also, this tutorial assumes that you're using Linux (in my case Ubuntu 16.04) but it should work on any Linux system.

This tutorial is taken from <https://github.com/pfalcon/esp-open-sdk>, please consider using their tutorial to build the complete toolchain, it provides more options like building SDK with less binary blobs.

I used Ubuntu 16.04, so the installation

```
$ sudo apt-get install make unrar autoconf automake libtool gcc g++ gperf \
flex bison texinfo gawk ncurses-dev libexpat-dev python-dev python \
python-serial \
sed git unzip bash help2man wget bzip2 libtool-bin
```

Later Debian/Ubuntu versions may require:

```
$ sudo apt-get install libtool-bin
```

▼ Pages 4

Find a page...

► Home

► Espusb on the Wemos D1 mini

► Example Projects

▼ Getting Started Guide

Table of Content

Hardware Schematics diagram

Building/Flashing using Espressif SDK

Building the toolchain

Building & flashing espusb

Editing the user.cfg

Building website

Flashing website to ESP

Building/flashing under a Docker container

Run HID example

+ Add a custom sidebar

Clone this wiki locally

https://github.com/cnlohr/espusb.'

Create an esp8266 folder, just to be neat:

```
mkdir ~/esp8266/  
cd ~/esp8266/
```

Git clone the esp-open-sdk and make it:

```
git clone --recursive https://github.com/pfalcon/esp-open-sdk.git  
cd esp-open-sdk/  
make  
export PATH=~/esp8266/esp-open-sdk/xtensa-lx106-elf/bin:$PATH
```

WARNING: ESPUSB doesn't work with ESP8266_NONOS_SDK v2.0 and above yet.
Please download and use v1.5.4 and below.

Download the ESP8266 nonos sdk v1.5.1 from espressif & delete the sdk provided
because the provided sdk is v2.0 and above:

```
rm sdk  
cd ..  
wget --content-disposition "http://bbs.espressif.com/download/file.php?id=1046"  
unzip ESP8266_NONOS_SDK_V1.5.1_16_01_08.zip
```

Finally, enter esp-open-sdk and link the sdk v1.5.x back.

```
cd esp-open-sdk/  
ln -sf ../esp_iot_sdk_v1.5.1 sdk
```

Building & flashing espusb

Install libusb dependency

```
sudo apt-get install libusb-1.0-0-dev
```

Git clone recursive from the espusb project and enter 'espusb' folder

```
git clone --recursive https://github.com/cnlohr/espusb  
cd espusb
```

Editing the user.cfg

Inside the user.cfg file you'll need to 'take care' of several variables:

- CHIP: 8266/8285
- IP: IP of the ESP (default: 192.168.4.1)
- PORT: the SERIAL port location of your ESP (default: /dev/ttyUSB0) check using dmesg
- PAGE_OFFSET: size of memory allocated for website
- SDK_DEFAULT: path location of your SDK (in our case: ~/esp8266/esp-open-sdk)
- FWBURNFLAGS: speed in which esptool flashes the binary into your ESP.
- SDK: location of the SDK folder (default is commented, remove the '#' and change the directory to "\$ (HOME)/esp8266/esp_iot_sdk_v1.5.1"

Sample of how user.cfg is configure for this.

```
#####  
# User Options
```

```
#####

#CHIP  = 8285
CHIP   = 8266
IP      = 192.168.4.1    # does not actually set the IP in firmware
PORT    = /dev/ttyUSB0   # could also be /dev/ttyACM0
WEB_PORT    = 80
COM_PORT   = 7777
BACKEND_PORT = 7878
PAGE_OFFSET = 65536      # 1048576

SDK_DEFAULT = $(HOME)/esp8266/esp-open-sdk
ESP_GCC_VERS = 4.8.5
FWBURNFLAGS = -b 1000000    #Change this to 115200 for CP2102
SDK          = $(HOME)/esp8266/esp-iot-sdk_v1.5.1

OPTS += -DICACHE_FLASH
#OPTS += -DVERIFY_FLASH_WRITE
#OPTS += -DDEBUG
#OPTS += -DFREQ=12500

SLOWTICK_MS = 100

PAGE_TITLE = esp8266 usb demo
PAGE_SCRIPT = <script language="javascript" type="text/javascript"
src=main.js></script>
PAGE_HEADING = $(PAGE_TITLE)
PAGE_INFO = <p>Welcome to the usb demo Web-based GUI.</p>
```

(BEWARE: CP2102 cannot flash on default value of 1000000, please change into 115200 or 1500000 ISSUE: <https://github.com/cnlohr/espusb/issues/8>)

Finally, run make all to compile the binary.

```
export ESP_ROOT=~/.esp8266/esp-open-sdk/"
make all
make burn
```



Building website

Ideally, the best way to flash the website would be to have the ESP8266 and the development PC to be in the same network. That way, whenever you run `make netweb` it'll flash over the WiFi to the ESP saving you time and effort from resetting the ESP.

Case #1: If your ESP and development PC is in the same network Run `make netweb` and it'll find the ip address based on what was set in user.cfg and flash to ESP.

```
make netweb
```



Case #2: If your ESP and development PC is NOT in the same network Run `make netweb` and then break once you start seeing Erase: 1 by hitting "Ctrl + C" to stop the program from attempting to flash over the network.

```
make netweb
```



--- wait till you see "Erase: 1" then hit Ctrl + C---

```
--- SAMPLE OUTPUT: ---
tmp/index.html: 4608 (4608)
tmp/menuinterface.js: 27648 (32256)
tmp/main.js: 6400 (38656)
tmp/jquery-2.1.4.min.js.gz: 29440 (68096)
256 68096
```



```
./pushtodev 192.168.4.1      65536      page.mpfs
Erase: 1
^CMakefile:30: recipe for target 'push' failed
make: *** [push] Interrupt
--- END of SAMPLE ---
```

You should now have a file called "page.mpfs" inside your web folder. If you have that, then your website has been compiled.

Flashing website to ESP

Once you have compiled the website, just burn the website over by going back to the root directory and make burnweb.

```
cd ~/espusb
make burnweb
```



Building/flashing under a Docker container

First install Docker and git on your system (and updated)

[Ubuntu docker](#), [docker-esp-sdk](#) This one worked for me:

Build the docker image (takes about 60-90 minutes)

```
git clone https://github.com/T-vK/docker-esp-sdk.git
cd docker-esp-sdk
sudo docker build -t tavk/esp-sdk:0.1.0
```



Instead of building it you could also just download it, if you trust the author:

```
sudo docker pull tavk/esp-sdk:0.1.0
```

Now assuming that your esp is connected at `/dev/ttyUSB0` and you cloned the espusb project to `/home/debian/espusb`, you can build and flash espusb by simply running this command:

```
sudo docker run -t -i -u esp \
--device=/dev/ttyUSB0 \
-v /home/debian/espusb:/home/esp/shared_project \
-e SDK_VERSION='1.5.3' \
tavk/esp-sdk:0.1.0 \
make ESP_ROOT=/home/esp/esp-open-sdk burn
```



You could of course put this long command in a bash script to make your life a bit easier.

To flash the web interface, you'll have to go inside the docker image as root (that won't be necessary anymore in the near future though):

```
sudo docker run -t -i -u root \
--device=/dev/ttyUSB0 \
-v /home/debian/espusb:/home/esp/shared_project \
-e SDK_VERSION='1.5.3' \
tavk/esp-sdk:0.1.0 \
bash
apt-get install libusb-1.0-0-dev
cd shared_project
make ESP_ROOT=/home/esp/esp-open-sdk burnweb
```



Other dockers: [Ubuntu docker](#), [esp8266-docker-buildbox](#) [Ubuntu docker](#), [vowstar](#)

Run HID example

Take a USB cable and connect your USB header that you made based on the hardware schematic diagram above. After connecting to your computer you'll see an output that

schematics diagram above. After connecting to your computer, you'll see an output that looks like this.

```
[32035.896810] usb 3-1: Product: ESPUSB2
[32035.896816] usb 3-1: Manufacturer: CNLohr
[32035.897447] usb 3-1: ep 0x81 - rounding interval to 64 microframes, ep desc says 80 microframes
[32035.897463] usb 3-1: ep 0x82 - rounding interval to 64 microframes, ep desc says 80 microframes
[32035.913561] input: CNLohr ESPUSB2 as /devices/pci0000:00/0000:00:1c.3/0000:09:00.0/usb3/3-1/3-1:1.0/0003:ABCD:8266.0006/input/input18
[32035.914363] hid-generic 0003:ABCD:8266.0006: input,hidraw5: USB HID v1.10 Mouse [CNLohr ESPUSB2] on usb-0000:09:00.0-1/input0
[32035.926449] input: CNLohr ESPUSB2 as /devices/pci0000:00/0000:00:1c.3/0000:09:00.0/usb3/3-1/3-1:1.1/0003:ABCD:8266.0007/input/input19
[32036.026139] hid-generic 0003:ABCD:8266.0007: input,hidraw6: USB HID v1.10 Keyboard [CNLohr ESPUSB2] on usb-0000:09:00.0-1/input1
```



If you got this message that means it works and your computer managed to read the ESP as a USB device.

Take your Android/PC/iPhone and open the WiFi settings and connect with your ESP device. *TIPS: The SSID of your ESP should look something like this 'ESP_A52378'*

Once connected, open your browser and surf into the IP address written inside your 'user.cfg'. By default it's `192.168.4.1`.

Once you're inside the website, you can tap onto the green zone to control the mouse and input your keyboard strokes.

Enjoy.

[+ Add a custom footer](#)